Domain 1: Agentic Architecture & Orchestration

A. Foundations of Agentic Architecture

What is Agentic Architecture?

Agentic architecture refers to a system design where Claude is given a goal, a set of instructions, access to tools, and enough context to determine what action should be taken next. Instead of only generating a single response, Claude can reason about the task, select an appropriate tool, inspect the tool results, update its understanding, and continue working until the task is complete.

In a traditional application, the developer usually defines every step in advance. For example, a support workflow might always run the same sequence: identify the customer, look up the order, check the refund policy, and then process a refund. This approach works well when requirements are stable and decisions are deterministic. However, it becomes less effective when the request is ambiguous or when the correct next step depends on new information discovered during the workflow.

In an agentic workflow, Claude can adapt to the situation. If the customer provides an order number but no account information, the agent may first verify the customer. If the lookup tool returns multiple matching accounts, the agent may ask a clarifying question. If the order is not eligible for an automatic refund, the agent may escalate to a human. This ability to reason over intermediate results is what makes the architecture agentic.

Anthropic describes the Agent SDK loop as a process where Claude receives the prompt, tool definitions, system prompt, and conversation history, evaluates the current state, requests tools when needed, receives tool results, and repeats until it produces a response with no tool calls.

Core Components of an Agentic System

A production-ready agentic system usually consists of several cooperating parts. Each part plays a specific role in helping Claude understand the task, take appropriate action, and maintain reliability.

User Request or Task Objective

The user request defines the goal that the agent must complete. It may be simple, such as "summarize this file", or complex, such as "investigate this billing dispute and determine whether the customer is eligible for a refund."

In an agentic architecture, the task objective should be clear enough for Claude to reason about success. Ambiguous requests may require clarification, tool use, or decomposition into smaller subtasks.

Example:

Customer request: "I was charged twice, my order arrived damaged, and I want this resolved today."

This is not a single-action request. A well-designed agent should recognize that it contains multiple concerns: billing, order condition, refund or replacement eligibility, urgency, and possible escalation.

System Prompt and Behavioral Instructions

The system prompt defines the agent's role, responsibilities, boundaries, and behavior. It tells Claude what kind of assistant it is, what tools it may use, how to handle uncertainty, and when to escalate.

For a customer support agent, the system prompt may specify that the agent should verify identity before accessing sensitive records, use backend tools for ordering details, follow company policy, and escalate when policy is ambiguous. For a developer productivity agent, the system prompt may define how Claude should explore files, run tests, and avoid destructive actions, and summarize changes.

System Prompt: Developer Agent

System prompts are important, but they should not be used as the only enforcement mechanism for high-risk operations. If a rule must always be followed, such as blocking refunds above a certain amount, use programmatic enforcement through hooks, prerequisite gates, or permission controls.

Claude Model Response

Claude's response can include natural language, tool-use requests, or both. In agentic systems, Claude does not simply answer immediately. It may first determine that it needs more information and then request a tool call.

For example, if a user asks, "Why did my order not arrive?", Claude should not guess. It should use an order lookup tool or shipment tracking tool if available. After receiving the result, Claude can reason about the next step and provide a grounded answer.

Tool Definitions

Tools define the actions Claude can request. A tool may loop up a customer, search files, read documents, run tests, process refunds, fetch policy data, query a database, or call an MCP server.

Anthropic's Agent SDK documentation lists built-in tools used by Claude Code and the Agent SDK, including file tools such as Read, Edit, and Write; search tools such as Glob and Grep; execution tools such as Bash; web tools such as WebSearch and WebFetch; and orchestration tools such as Agent, Skill, and task-tracking tools.

Clear tool definitions are essential because Claude uses tool names, descriptions, and schemas to decide which tool is appropriate. Poorly described tools can lead to incorrect tool selection, duplicate work, or unsafe actions.

Built-in tools

The SDK includes the same tools that power Claude Code:

Category	Tools	What they do
File operations	Read, Write, Edit	Read, modify, and create files

Category	Tools	What they do
Search	Glob, Grep	Find files by pattern, search content with regex
Execution	Bash	Run shell commands, scripts, git operations
Web	WebSearch, WebFetch	Search the web, fetch and parse pages
Discovery	ToolSearch	Dynamically find and load tools on-demand instead of preloading all of them
Orchestration	Agent, Skill, AskUserQuestion, TaskCreate, TaskUpdate	Spawn subagents, invoke skills, ask the user, track tasks

Beyond built-in tools, you can:

- Connect external services with MCP servers (databases, browsers, APIs)
- Define custom tools with custom tool handlers
- Load project skills via setting sources for reusable workflows

Tool Execution Layer

The tool execution layer is the part of the application that actually runs the tool Claude requested. Claude decides which tool it wants to use, but the application controls whether the tool call is allowed and how the result is returned.

This separation is important. Claude may request a tool, but the system should still apply permission checks, policy rules, validation, and logging before executing it. Anthropic's documentation states that Claude determines which tools to call based on the task, but developers control whether those calls are allowed to execute through settings such as allowed tools, disallowed tools, and permission modes.

Tool Results

Tool results provide Claude with new information. These results must be returned to the conversation context so Claude can reason about the next action. A tool result may include customer data, order status, file contents, test output, policy information, or an error message.

Tool results should be concise, structured, and relevant. Verbose results can consume unnecessary context, while incomplete results may cause Claude to make unsupported assumptions.

Example: Useful tool results (JSON)

```
{
"customer_id": "C-1049",
"verification_status": "verified",
"order_id": "O-88421",
"order_status": "delivered",
"issue": "damaged item",
"photo_evidence": true,
```

```
"refund_limit_check": "requires_human_approval"
}
```

This result is easier for Claude to reason over than a large raw database response containing dozens of irrelevant fields.

Conversation History

Conversation history contains the prior user messages, Claude responses, tool calls, and tool results. It gives Claude continuity across turns.

In an agentic workflow, conversation history is especially important because Claude may need to remember what it already checked, which tools have already returned results, what assumptions are valid, and what still needs to be done.

However, conversation history also grows over time. Long-running agents can accumulate too much context, especially when they read large fuels, run verbose commands, or use tools repeatedly. Anthropic's Agent SDK documentation explains that the context window includes the system prompt, tool definitions, conversation history, tool inputs, and tool outputs, and that large tool outputs can consume significant context within Claude's native token limit. handles the context window by automatically managing session histories, tool inputs, and tool outputs within Claude's native token limit.

While standard Claude models operate within a baseline 200,00000-token context window, advanced setups like Claude Code or specific model tiers (e.g., Claude Opus variants) can scale up to a 1 million-token context window. Because long-running agent tasks can quickly overflow this window with verbose tool results, the Agent SDK provides active strategies to maintain context

Control Loop

The control loop is the mechanism that repeatedly sends the updated context back to Claude until the task is complete. In Claude API terms, this involves checking whether Claude wants to use a tool or whether Claude wants to use a tool or whether it has finished its turn.

For the CCA-F exam, this concept becomes more important in the next section, Agentic Loop Lifecycle, where you will encounter `stop_reason`, `tool_use`, and `end_turn` handling in more detail. The key point here is that an agentic system is not a single request-response interaction. It is a repeated reasoning and action cycle.

Stop Condition

The stop condition determines when the agent should end the loop and return a final response. The official exam guide emphasizes that agentic loop control should continue when `stop_reason` is "tool_use" and terminate when `stop_reason` is "end_turn"; it also warns against using unreliable anti-patterns such as parsing natural language completion signals or checking for assistant text as the completion indicator. A good architecture uses explicit control signals and avoids guessing whether Claude is done.

Session State

Session state allows the system to preserve continuity across longer workflows. This is important when an agent needs to continue an investigation, resume a prior session, or fork an earlier analysis into multiple possible solution paths.

In Domain 1, candidates are expected to understand session resumption and forking. These concepts are especially useful for codebase exploration, research workflows, and alternative implementation planning.

Error Handling and Escalation Logic

Error handling defines what happens when a tool fails, returns incomplete data, or produces an ambiguous result. Escalation logic defines when the agent should stop autonomous work and hand the case to a human.

In a production system, the agent should not treat every tool failure the same way. A timeout may be retryable, a permission error may require escalation, and a policy exception may require human review. For customer support scenarios, escalation should include a structured handoff summary so the human agent can act without re-reading the entire conversation.

Agentic System vs Traditional Workflow

Aspect	Agentic System	Traditional Workflow
Decision-making	Model-driven, Claude reasons about next action	Predefined logic, application follows preconfigured steps
Tool selection	Model selects tools based on context	Application invokes tools by rule
Adaptability	High	Limited
Flexibility	High	Low to moderate
Best for	Ambiguous, multi-step, exploratory tasks	Predictable and deterministic tasks
Risk	Requires guardrails and monitoring	Easier to validate upfront
Human review	Often integrated	Usually separate

EXAM TIP: If the scenario describes ambiguity, multi-step investigation, or changing decisions based on tool results, agentic architecture is usually appropriate. If the scenario describes mandatory compliance ordering, programmatic enforcement is usually required.

For example, a traditional workflow might always execute:

1. Look up customers.
2. Look up the order.
3. Check refund policy.
4. Process refund.

An agentic workflow can adapt:

1. Ask for missing customer information if multiple matches are found.

2. Look up the order only after verification.
3. Investigate billing, shipping, and refund issues separately if the customer raises multiple concerns.
4. Escalate to a human if policy is ambiguous.
5. Process the refund only if all prerequisites are satisfied.

A traditional workflow is best when the process is stable, deterministic, and easy to define in advance. Examples include sending a password reset email, formatting a known data field, or running a scheduled report.

An agentic workflow is better when the task requires investigation, context-sensitive judgement, multiple tools, or adaptive next steps. Examples include researching a broad topic, debugging a failing test suite, triaging a customer complaint, or exploring an unfamiliar codebase.

The Agent SDK documentation describes the loop as Claude evaluating a prompt, calling tools, receiving tool results, and repeating until the task is complete: https://code.claude.com/docs/en/agent-sdk/agent-loop

When Not to Use Agentic Architecture

Do not use agentic architecture when the task is simple, deterministic, and better handled by fixed application logic.

Examples include:

- A static form validation rule
- A schedule data export
- A simple notification workflow
- A direct database lookup with no ambiguity
- A required compliance check that must always run
- A fixed approval process with no adaptive reasoning

For these cases, traditional application logic may be safer, cheaper, easier to test, and easier to audit.

Common CCA-F use cases include:

- Customer support resolution agents
- Multi-agent research systems
- Developer productivity agents
- Codebase exploration agents
- Automated test-generation workflows
- Structured data extraction pipelines
- CI/CD review assistants

Key Design Considerations

- **Guardrails:** because agentic systems can take actions, they need guardrails. Guardrails may include tool permissions, hooks, structured validation, human approval, and escalation rules.
- **Tool Scope:** an agent should not have every possible tool by default. Too many tools can increase decision complexity and create safety risks.

- **Observability:** a production agent should be observable. The system should record which tools were called, what results were returned, when an escalation occurred, and why the final decision was made.
- **Escalation:** an agent should know when not to continue. Escalation is appropriate when user intent is unclear, policy is ambiguous, required information is missing, tool failures prevent meaningful progress, or the user explicitly asks for a human.
- **Context Management:** long-running agents must manage context carefully. Verbose tool results, repeated file reads, and unstructured summaries can make the session harder to reason over. Use concise tool results, structured facts, and subagents where appropriate.

Common Anti-Patterns

Avoid these design mistakes:

- Treating every Claude application as an agentic system
- Giving the agent too many tools without restrictions
- Relying only on prompts for mandatory compliance rules
- Allowing state-changing tools without approval or validation
- Returning large raw tool outputs to Claude without filtering
- Designing agents that cannot escalate
- Assuming subagents automatically inherit all parent context
- Using agentic workflows for simple deterministic processes
- Failing to log tool use and decisions
- Letting an agent continue indefinitely without limits or budget controls

---

B. Agentic Loop Lifecycle

Agent Loop Definition

An agentic loop is the repeated cycle in which Claude receives a task, reasons about the current state, decides whether to call a tool, receives the tool result, and continues until the task is complete. In a simple one-turn interaction, Claude may answer directly. In an agentic workflow, Claude may need several reasoning and tool-use turns before producing a final answer.

Antropic's Agent SDK documentation describes the agentic loop as the same autonomous execution cycle that powers Claude Code. In that cycle, Claude receives the prompt together with the system prompt, tool definitions, and conversation history; evaluates the current state; requests tools when needed; receives the results; and repeats until the task is complete. The SDK documentation presents the loop in five stages: receive a prompt, evaluate and respond, execute tools, repeat, and return a result.

Example pattern:

```
User Request:
Fix the failing authentication tests.

Claude:
Runs the test suite.
```

```
Tool Result:
Three authentication tests failed.

Claude:
Read the relevant source and test files.

Tool Result:
auth.ts and auth.test.ts are returned.

Claude:
Edits the source file.

Tool Result:
File updated.

Claude:
Run the tests again.

Tool Result:
All tests passed.

Claude:
Returns the final summary.
```

This pattern is important for CCA-F because many exam scenarios involve workflows where Claude must decide the next action based on intermediate tool results, not merely follow a fixed script.

The most important conceptual point is that an agentic loop is iterative. A Claude-powered agent is not a one-shot request-responses system if tools are involved. A single task may require several turns, and each turn changes what Claude knows. Anthropic defines a turn as one round trip in which Claude produces output that include tool calls, the SDK runs those tools, and the results feed back into Claude automatically. The loop ends only when Claude produces output with no tool calls.

This distinction is central to the CCA-F exam's idea of "autonomous task execution". In a traditional fixed workflow, the application already knows the sequence of operations. In an agentic loop, the application provides Claude with tools and context, and Claude decides what to do next after seeing intermediate results. That is why the loop is treated as an architectural pattern rather than a mere programming convenience.

Stop Reasons as Control Signals

The `stop_reason` is the control signal that tells your application why Claude stopped generating a given response. Anthropic's Messages API documentation states that `stop_reason` is part of every successful response and indicates why Claude completed generation, as distinct from an API failure.

When `stop_reason` is "tool_use", Claude is not finished. It is explicitly asking the application to execute one or more client-side tools. Anthropic's documentation says that in this case Claude is "calling a tool and expects you to execute it." In practical terms, "tool_use" means "continue the loop." You must examine the "tool_use" blocks, run the requested operation, and then return the result to Claude so it can decide what to do next.

When `stop_reason` is "end_turn", Claude has finished its response naturally. Anthropic labels this as the most common stop reason and shows it as the signal for processing a complete response. In practical terms, "end_turn" means "stop the loop and return the final answer," unless another documented stop reason such as `max_tokens`, `pause_turn`, or `model_context_window_exceeded` applies. For Domain 1, however, the official exam guide explicitly singles out the "tool_use" versus "end_turn" distinction as required knowledge and skill.

Example (Python):

```
if response.stop_reason == "tool_use":
    # execute tool(s), append results, continue loop
elif response.stop_reason == "end_turn":
    # final response, terminate loop
else:
    # handle other documented stop reasons appropriately
```

That logic is consistent with both Anthropic's stop-reason guidance and the official exam blueprint. One nuance worth knowing is that Anthropic documents an implementation hazard around "end_turn" after tool results. If tool results are formatted incorrectly, especially if extra text is inserted in the wrong place, Claude can return an empty response with `stop_reason: "end_turn"` because it interprets the assistant turn as already complete. That is not a signal that the model "failed to think"; it is often a message-formatting bug.

Returning Tool Results and Updating Conversation History

The CCA-F exam guide states that tool results must be appended to conversation history so the model can reason about the next action. Anthropic's SDK documentation says the same thing in operational terms: each set of tool results feeds back to Claude for the next decision. Without that update, there is no real loop.

Anthropic distinguishes between client tools and server tools. For client tools, Claude returns `stop_reason: "tool_use"` together with one or more `tool_use` blocks; your application executes the tool and sends back a `tool_result`. For server tools such as `web_search`, `web_fetch`, `code_execution`, and `tool_search`, Anthropic executes the tool on its own infrastructure and integrates the results directly into the response. This distinction matters because only the client-tool path requires your application to manually continue the conversation with a `tool_result`.

In a manual Messages API implementation, the documented sequence is precise. After receiving a client tool request, you extract the tool's name, id, and input; execute the tool in your own code; then continue the conversation with a new user message containing a `tool_result` block that references the original `tool_use_id`. Anthropic further specifies two formatting requirements: the tool result must immediately follow the assistant's tool-use message in the message history, and within the user message the `tool_result` blocks must come first in the content array, with any text appearing only after all tool results. If you violate that ordering, Anthropic warns that the API can return a 400 error.

That requirement explains why the official exam guide emphasizes "conversation history updates" rather than merely "tool execution." The architectural unit is not "Claude called a tool." The architectural unit is "Claude called a tool, the tool executed, and the result was reintroduced into the structured conversation

so Claude could reason from it." A loop that executes tools but does not return their results is incomplete and unreliable by design.

Anthropic also documents a particularly relevant mistake: adding free text immediately after `tool_result` can lead to empty "end_turn" responses because Claude learns that the user will provide explanatory text instead of expecting the model to continue reasoning. The safer pattern is to send tool results directly, with no unnecessary narrative inserted between the tool use and the tool result.

Example of documented manual loop pattern (Python):

```
messages = [{"role": "user", "content": user_query}]
while True:
    response = client.messages.create(
        model="claude-opus-4-8",
        max_tokens=1024,
        messages=messages,
        tools=tools
    )
    if response.stop_reason == "tool_use":
        tool_results = execute_tools(response.content)
        messages.append({"role": "assistant", "content": response.content})
        messages.append({"role": "user", "content": tool_results})
    elif response.stop_reason == "end_turn":
        return response
    else:
        return handle_other_stop_reason(response)
```

This structure closely follows Anthropic's own manual examples for tool workflows and stop-reason handling.

Model-Driven Orchestration Versus Preconfigured Flows

A major conceptual distinction in Task Statement 1.1 is the difference between model-driven tool selection and pre-configured flows. The exam guide states this directly: Claude should reason about which tool to call next based on context, rather than simply following a pre-wired decision tree or hard-coded tool sequence.

Anthropic's tool-use documentation explains this behavior through `tool_choice`. With the default setting of `{"type": "auto"}`, Claude decides on each turn whether to call a tool or respond directly. Anthropic says Claude calls a tool when the request maps to that tool's described capability and the answer is not already in context; it answers directly for stable knowledge, creative tasks, and ordinary conversational turns. In other words, tool invocation under auto is contextual and model-driven, not mechanically predetermined.

This is what makes an agentic loop different from a scripted pipeline. In a hard-coded flow, the application might always call `get_customer`, then `lookup_order`, then `process_refund`, regardless of what the intermediate results show. In a model-driven loop, Claude can inspect the first result and decide that a clarifying question is needed, that a different tool is relevant, or that escalation is more

appropriate than another backend operation. That adaptive reasoning is exactly what the exam is testing when it contrasts agentic loops with preconfigured sequences.

At the same time, model-driven does not mean uncontrolled. Anthropic documents that you can nudge or constrain behavior with prompt wording and `tool_choice`, and other exam task statements make clear that deterministic rules should be enforced programmatically when compliance or safety requires it. For this subsection, the key architectural lesson is simpler: use the loop to let Claude choose the next best action from context, but do not confuse that flexibility with a license to skip required controls elsewhere in the system.

Anti-Patterns and Implementation Guidance

The exam guide is unusually explicit about what not to do. It warns against parsing natural-language signals to determine loop termination, using arbitrary iteration caps as the primary stopping mechanism, and checking assistant text content as a completion indicator. These are anti-patterns because they try to infer control state from surface output rather than from the documented contract of `stop_reason`. If Claude writes "I'm done," that is not the canonical signal. The canonical signal is `stop_reason`.

Anthropic's stop-reason guidance reinforces that best practice by explicitly recommending that developers always check `stop_reason` in response-handling logic. The docs even provide example branching logic that handles "tool_use" separately from "end_turn" and other stop reasons. This is the correct mental model for CCA-F: loop control should be driven by structured protocol fields, not by heuristics over assistant prose.

Another subtle anti-pattern is to treat `max_turns` as the primary definition of task completion. Anthropic's Agent SDK provides `max_turns` and budget controls for production safety, and the SDK documentation explains that `max_turns` counts tool-use turns only. Those controls are useful guardrails for cost and runaway behavior, but they are not the semantic signal that the task is complete. The loop's true completion condition is still Claude producing a response with no further tool calls, typically reflected as "end_turn" in a manual API loop or a final text-only assistant response in the SDK.

Formatting mistakes are another class of anti-pattern because they masquerade as reasoning failures. Anthropic states that tool results must immediately follow the corresponding tool-use message and must appear first in the user content array; otherwise the API may reject the request. The same documentation warns that adding text after tool results can provoke empty "end_turn" responses. From a system-design point of view, these are not cosmetic issues. Message ordering is part of the execution protocol.

A sound exam-ready rule of thumb is therefore this: use `stop_reason` to decide whether to continue or terminate; use `max_turns` and budget as safety rails, not completion detectors; preserve assistant `tool_use` and user `tool_result` messages in the correct order; and never rely on natural-language phrasing or vague assistant text as your loop-control mechanism.

C. Coordinator-Subagent Orchestration

Coordinator as hub and subagents as spokes

The official exam guide describes the recommended architecture as hub-and-spoke: a coordinator agent sits at the center and manages inter-subagent communication, error handling, and information routing. It

is the authority that determines what work exists, what work has been completed, and how partial findings should be merged into a final answer.

Anthropic's subagents documentation supports this model technically. It explains that subagents are distinct agent instances designed for focused subtasks, and it highlights three benefits that matter directly to orchestration design: context isolation, parallelization, and specialized instructions. A subagent can therefore be treated as a role-bounded worker rather than as a second copy of the parent agent. This is what allows a coordinator to assign one agent to current web sources, another to document analysis, and another to synthesis without crowding the main conversation with all intermediate reasoning.

A further implementation detail strengthens the hub-and-spoke reading: in the current SDK, subagents cannot spawn their own subagents. Anthropic explicitly instructs developers not to include the Agent tool in a subagent's tool list. In practice, that means the coordinator remains the single orchestrator, which simplifies observability, permissions, and failure handling. Architecturally, this is useful because it prevents uncontrolled nested delegation trees and keeps ownership of workflow state at the center.

This design should be understood as a deliberate contrast with a peer-to-peer multi-agent mesh. A mesh may appear flexible, but it obscures provenance, complicates retries, and makes it difficult to determine which agent had authority over which decision. The exam guide's emphasis on routing all subagent communication through the coordinator reflects the opposite priority: controlled information flow, consistent recovery logic, and auditability.

Context isolation and explicit handoff design

One of the most important exam facts in this area is that subagents do not automatically inherit the coordinator's conversation history. Anthropic's documentation states that each subagent starts in its own fresh conversation, and that the only direct channel from parent to subagent is the Agent tool's prompt string. The subagent does not receive the parent's conversation history or tool results unless those are explicitly included in that prompt.

This has major architectural consequences. A coordinator cannot delegate by saying, in effect, "analyze the issue from earlier." It must hand off the relevant facts directly: the problem statement, the scope of work, the specific files or sources to inspect, the constraints, and any findings already produced by other agents. If the coordinator omits those elements, the system does not fail gracefully; it fails by giving the subagent an underspecified problem and inviting hallucinated assumptions. The exam guide turns this into a tested skill by requiring candidates to understand that context must be passed explicitly and that subagents operate with isolated state.

Anthropic's docs add another important nuance: the parent does not receive the subagent's full internal transcript. Intermediate tool calls and tool results stay inside the subagent, and only the subagent's final message returns to the coordinator as the Agent tool result. This is excellent for keeping the main context window smaller, but it also means that if the coordinator needs provenance, citations, timestamps, or structured metadata for later synthesis, the subagent's final output must include those explicitly. Otherwise, the coordinator receives a clean summary but loses the traceability needed for downstream reasoning.

For exam purposes, the correct design implication is straightforward: when subagents contribute evidence that will later be merged, their outputs should be structured, bounded, and attribution-

preserving. The coordinator should treat every subagent return as a formal handoff artifact, not as a casual chat reply. That principle also anticipates adjacent exam topics on context passing and provenance preservation.

Dynamic delegation, partitioning, and synthesis

The exam guide is explicit that a coordinator should dynamically select which subagents to invoke, rather than always routing every request through a fixed pipeline. This matters because not all tasks justify the same orchestration cost. A simple request may need only one specialized worker; a broad investigative request may need several. The core design judgment is therefore selective delegation, guided by the shape of the question and the type of evidence required to answer it.

Current Anthropic documentation provides the mechanism for that behavior. Claude decides whether to invoke a subagent based primarily on the subagent's description field, and developers can also force a specific subagent by naming it explicitly in the prompt. The same docs also show that agent definitions can be created dynamically at runtime, which means a coordinator can vary prompt strictness, model choice, or tool scope depending on the job. These details are architecturally important because they show that "dynamic delegation" is not only an exam concept; it is directly supported in the SDK.

Partitioning strategy is the second half of delegation quality. The exam guide warns that overly narrow decomposition by the coordinator produces incomplete coverage of broad topics. The guide's own sample scenario makes this concrete: when asked to research the impact of AI on creative industries, a flawed coordinator decomposed the topic into "digital art creation," "graphic design," and "photography," causing the final report to miss music, writing, and film. The problem was not that the subagents performed poorly; it was that the coordinator created the wrong decomposition boundary.

A sound coordinator therefore partitions work in ways that are both distinct and collectively sufficient. In practice, that often means splitting by source type (web sources versus documents), by domain slice (music, writing, film, visual arts), or by analytic function (search, analysis, synthesis, report generation). Anthropic's subagent docs support these designs by emphasizing specialized prompts, parallel subagent execution, and restricted tool sets matched to each role. A document reviewer may need Read, Grep, and Glob; a test-runner may need Bash; a synthesis agent may need no wide-open search tools at all.

The coordinator's aggregation role is just as important as its delegation role. After receiving subagent outputs, it must reconcile overlaps, identify contradictions, remove duplication, and determine whether the combined answer satisfies the user's scope. In a research system, that means verifying that the final synthesis covers the requested landscape rather than merely assembling four independent summaries into one longer document. The exam guide describes this as result aggregation plus a decision about whether additional subagent work is needed.

Iterative refinement, observability, and scale boundaries

The strongest version of this architecture is not one-pass delegation. It is an iterative refinement loop. The exam guide specifically calls for a coordinator that evaluates synthesis output for gaps, re-delegates targeted follow-up queries to search or analysis agents, and then re-invokes synthesis until coverage is sufficient. This is one of the clearest exam signals that orchestration is about supervision, not one-time fan-out.

In practical terms, the loop is straightforward. The coordinator decomposes the task, launches the relevant subagents, gathers their results, performs a coverage review, and asks whether any required dimension remains thin, missing, or weakly supported. If the answer is yes, it sends targeted follow-up tasks rather than rerunning the entire workflow blindly. That pattern improves both quality and cost control because it narrows the second pass to the actual gap instead of repeating already-sufficient work. The exam guide treats this targeted redelegation pattern as a tested skill.

Routing all communication through the coordinator is also the cleanest way to maintain observability. Since only the coordinator sees the full task graph, it is the only agent that can reliably answer questions such as which subagents ran, which ones failed, whether there are unresolved gaps, and which findings reached the final synthesis. Anthropic's docs reinforce this by showing that subagents return only final messages to the parent and by exposing subagent invocation through `tool_use` blocks for the Agent tool. In other words, the coordinator is both the logical and the inspectable control point.

There is also an important production boundary. Anthropic notes that ordinary subagent delegation works well for a few delegated tasks per turn, but for workflows coordinating dozens to hundreds of agents, the recommended pattern is the Workflow tool, which moves orchestration outside the main conversation context. That recommendation is useful because it clarifies the scale at which coordinator–subagent interaction remains the right abstraction. For CCA-F, the tested pattern is still the conversational coordinator, but in production architecture discussions it is worth knowing where that pattern begins to strain.

Common pitfalls and authoritative sources

The most common failure mode in this domain is treating decomposition as a purely mechanical split rather than as a coverage design problem. The exam guide's "creative industries" example shows that if the coordinator frames the task too narrowly, every downstream component can succeed locally while the system fails globally. The second common mistake is assuming subagents share the parent's state implicitly; Anthropic's current docs explicitly say they do not. The third is letting every request traverse the entire pipeline regardless of complexity, even though the exam guide says the coordinator should invoke subagents dynamically based on need.

A fourth pitfall is over-permissive tool design. Anthropic's permissions documentation explains that `allowedTools` can be used to lock subagents down and that permissive modes such as `bypassPermissions` are inherited by subagents and cannot be relaxed per subagent. That matters in orchestration because a coordinator with broad autonomy can accidentally create a fleet of equally broad workers unless permissions and tool scopes are deliberately constrained. In a formal architecture review, that is a governance issue as much as a technical one.

D. Subagent Invocation & Context Passing

How subagents are spawned

In the current Claude Agent SDK, subagents are invoked through the Agent tool. Anthropic's official SDK examples say to include Agent in `allowedTools` so Claude can auto-approve subagent invocation without a permission prompt. The same page shows the standard programmatic pattern: define subagents in the `agents` parameter, then give the parent agent permission to invoke them by allowing

Agent. The docs also note that even if you do not define a custom subagent, Claude can still invoke the built-in general-purpose subagent through the Agent tool.

That is the modern SDK behavior, but the official docs also preserve the historical naming bridge. Anthropic states that the tool name was renamed from Task to Agent in v2.1.63, and that existing `Task(...)` references in settings and agent definitions still work as aliases. In other words, if a question or study note says a coordinator must include "Task" in `allowedTools`, the implementation idea is still the same: the parent agent must be permitted to spawn subagents. In current SDK examples, that permission is expressed as Agent; in older exam phrasing, it may still appear as Task.

There is a second important spawning nuance: current Anthropic guidance increasingly favors descriptive delegation over rigid hard-coding. The SDK docs say Claude decides when to invoke a subagent based on the task and the subagent's description, while explicit prompt mentions such as "Use the code-reviewer agent" force a specific delegation. That means a coordinator can either let Claude choose dynamically based on descriptions or explicitly name a specialized worker when correctness requires it.

What context a subagent does and does not receive

The single most important fact in this topic is that a subagent does not automatically inherit the coordinator's conversation history. Anthropic's SDK docs are explicit: a subagent starts with a fresh context window, and the only direct parent-to-subagent channel is the Agent tool's prompt string. The docs say the parent must include any file paths, error messages, decisions, or other information the subagent needs directly in that prompt. They also state that the subagent does not receive the parent's conversation history or tool results.

That does not mean the subagent starts from absolute zero. The current docs describe a more precise model: the subagent receives its own system prompt, the delegation message, and tool definitions; in Claude Code it can also receive project CLAUDE.md, memory, and some session environment information depending on the subagent type. Anthropic also documents an important exception: built-in Explore and Plan agents skip CLAUDE.md and git status, so if a project rule absolutely must reach those agents, you should restate it in the delegation prompt.

This has a very practical implication for CCA-F-style architectures. If the coordinator already has twelve findings from prior tools or prior agents, a prompt like "continue the earlier analysis" is weak, because the subagent cannot see what "earlier" means unless that content is passed explicitly. Anthropic's prompting guidance reinforces this: for complex tasks, the model performs better when instructions, context, examples, and variable inputs are clearly separated, ideally with structured tags. Combined with the subagent docs, the correct architectural conclusion is that context passing is an explicit handoff design problem, not an automatic memory feature.

The return path is also constrained. The parent conversation does not get the subagent's full internal transcript by default; it gets the subagent's final message as the agent tool result. Anthropic notes that the parent may then summarize that result further in its own response. If you need the exact wording preserved, the docs recommend telling the parent to preserve the subagent output verbatim. For orchestration design, this means the subagent's final report is the safest artifact to capture and pass forward to later workers.

How to define subagents well

Anthropic's current SDK documentation makes the AgentDefinition shape very clear. At minimum, a subagent should have a `description` and a `prompt`. The description tells Claude when to use the agent; the prompt defines the subagent's role, behavior, and expertise. Anthropic's examples then layer in restricted `tools`, optional `disallowedTools`, optional model overrides, skills, memory, MCP servers, `maxTurns`, `permissionMode`, and other controls. The key exam-aligned idea is simple: a subagent should be defined as a focused specialist with a clear job description and an appropriately narrow capability envelope.

The official docs also make tool restriction behavior explicit. If you omit the `tools` field, the subagent inherits all available tools by default. If you specify `tools`, the subagent is restricted to that allowlist. If you specify `disallowedTools`, those are removed from the inherited or explicit set. Anthropic further documents that if both are present, `disallowedTools` is applied first and then `tools` are resolved against what remains. This matters because exam questions often hide the real issue inside capability scope: a reviewer agent should not quietly inherit file write or shell execution if it only needs Read, Grep, and Glob.

A strong current-era design also uses descriptions strategically. Anthropic says Claude uses the description field to decide when to delegate and recommends writing clear, specific descriptions so tasks match the right subagent. That is why "expert code reviewer" is weaker than "expert code review specialist; use for quality, security, and maintainability reviews," and why "research agent" is weaker than "finds and summarizes policy evidence with citation metadata." The better the description, the less the coordinator has to micromanage.

There is one subtle but important version caveat here. The current subagent SDK docs still say "Subagents cannot spawn their own subagents" and advise not to include Agent in a subagent's tools array. But the official Claude Code changelog for June 10, 2026 says the opposite: "Sub-agents can now spawn their own sub-agents (up to 5 levels deep)." The most reasonable interpretation is that the changelog reflects a newly shipped capability and the conceptual docs have not fully caught up yet. For live systems, you should trust your installed version and recent changelog; for certification study, you should expect questions to follow the more stable principles in the guide you are studying unless the exam explicitly mentions the new behavior.

Forks, resumes, and parallel work

There are really two different "fork" ideas you need to distinguish. At the session level, the Agent SDK's session guide says "fork" creates a new session that starts with a copy of the original session's history while leaving the original unchanged. Anthropic recommends this when you want to try an alternative approach without losing the original path. That maps cleanly to the exam idea of exploring divergent approaches from a shared analysis baseline.

At the subagent level, Claude Code documents a special kind of forked subagent that inherits the entire conversation so far instead of starting from a fresh context window. Anthropic explains that this drops the normal input isolation, which is precisely why forks are useful when a named subagent would need too much background to be effective. The same docs say a fork sees the same system prompt, tools, model, and message history as the main session, while its own tool calls still stay out of the parent conversation and only its final result is returned. Anthropic also notes that a fork can use isolation: "worktree," so edits happen in a separate git worktree rather than the parent checkout.

Resumption is related but different. Anthropic's subagent SDK docs say resumed subagents retain their full conversation history, including prior tool calls and reasoning, and continue where they left off rather than starting fresh. The official pattern is to capture the parent `session_id`, extract the subagent's `agentId` from the Agent tool result, and then resume the same session with a prompt that names the agent to continue. Anthropic also documents that the built-in Explore and Plan agents are one-shot and do not return an `agentId`, so if you know you will need resumability, you should use a custom subagent or general-purpose instead.

Parallelism is the other major practical skill in this topic. Anthropic's SDK docs say multiple subagents can run concurrently so independent subtasks finish in the time of the slowest one rather than the sum of all of them. The Claude Code docs give the exact kind of example that maps well to exam scenarios: researching the authentication, database, and API modules in parallel using separate subagents. Anthropic's broader prompting guidance also explicitly recommends parallel tool calls when there are no dependencies between them. So when a study guide says a coordinator should emit multiple Task calls in one response to fan out independent work, the current public docs frame the same core idea as concurrent subagents and parallel tool execution.

E. Multi-Step Workflow Enforcement & Handoff

Programmatic Enforcement vs. Prompt Guidance

Prompt guidance tells Claude what it should do. Programmatic enforcement controls what the system allows Claude to do.

Prompt instructions are useful for tone, style, preferences, and general behavior. For example, a system prompt can instruct Claude to be concise, explain decisions clearly, or ask clarifying questions when information is missing. However, prompt instructions should not be the only control for operations that must never happen out of order.

Programmatic enforcement is required when a rule must be guaranteed. Examples include identity verification before financial actions, blocking refunds above a threshold, preventing unauthorized account changes, or requiring human approval before sensitive operations. The exam guide specifically identifies prerequisite gates and hooks as enforcement mechanisms for workflow ordering.

Requirement	**Recommended Control**
Use a friendly support tone	Prompt guidance
Ask clarifying questions when needed	Prompt guidance
Verify customer identify before refund	Programmatic prerequisite gate
Block `process_refund` before `get_customer` succeeds	Programmatic enforcement
Escalate refunds above a threshold	Hook or prerequisite gate
Summarize case details for a human agent	Structured handoff protocol

EXAM TIP: A common exam trap is choosing to "improve the prompt" when the workflow requires deterministic compliance. If the consequence of failure is financial, legal, security-related, or customer-impacting, a prompt alone is usually not sufficient.

Deterministic Compliance

Deterministic compliance means the system enforces a rule every time, regardless of how Claude interprets the prompt. In agentic workflows, Claude may decide which tool to call next based on context, but the application should still prevent invalid or unsafe tool calls.

For example, a support agent may have access to the following tools:

- `get_customer`
- `lookup_order`
- `process_refund`
- `escalate_to_human`

A prompt might say:

```
get_customer
lookup_order
process_refund
escalate_to_human
```

This is helpful, but it does not guarantee compliance. A stronger architecture adds a prerequisite gate:

Rule: Block `process_refund` unless `get_customer` has returned a verified customer ID.

If Claude attempts to call `process_refund` before customer verification, the system blocks the tool call and returns a controlled response, such as the following:

Example (JSON):

```
{
"allowed": false,
"reason": "Customer verification is required before processing a refund.",
"next_action": "Call get_customer or escalate_to_human."
}
```

Prerequisite Gate Pattern

A prerequisite gate is a programmatic rule that prevents a downstream action until one or more required upstream steps have completed successfully.

Example (Refund Workflow):

```
Step 1: get_customer
Required result: verified_customer_id

Step 2: lookup_order
Required result: valid order associated with verified customer
```

```
Step 3: check refund policy
Required result: eligible refund or escalation condition

Step 4: process_refund
Allowed only if all prerequisites are satisfied
```

In this pattern, Claude can still reason about the customer's issue, choose tools, and explain the resolution. However, it cannot bypass required verification steps.

Example (Gate Logic):

```
If process_refund is requested:
    Check whether verified_customer_id exists.
    Check whether order_id belongs to verified_customer_id.
    Check whether the refund amount is within policy.

    If checks pass:
        Allow process_refund.
    If checks fail:
        Block process_refund and return the required next action.
```

This makes the workflow reliable because the system enforces the rule rather than relying only on Claude to remember it.

Hooks for Enforcement

Hooks can also be used to intercept tool calls and enforce business rules. While prerequisite gates are often used to enforce step ordering, hooks are useful when the system must inspect or modify a tool call before it executes.

Examples include:

- Blocking refunds above $500
- Preventing changes to restricted account fields
- Requiring human approval for sensitive operations
- Blocking destructive file or database actions
- Normalizing or validating tool input before execution

The exam guide connects this concept to later Domain 1 topics by describing hooks as a way to enforce compliance rules, such as blocking refunds above a threshold, and choosing hooks over prompt-based enforcement when business rules require guaranteed compliance.

F. Agent SDK Hooks for Tool Interception & Data Normalization

Hooks are the primary mechanism for deterministic enforcement in Claude Code and the Agent SDK. They allow developers to insert application-layer logic at specific points in the tool execution lifecycle without relying on model compliance. Understanding hooks is essential for the CCA-F exam because

many scenarios test the ability to distinguish between what prompts can guarantee and what only hooks can guarantee.

Anthropic's documentation defines hooks as user-defined shell commands that execute at specific points in Claude's processing. They are not suggestions to the model, they run regardless of what the model decides. This makes them the correct enforcement mechanism whenever a rule must apply on every tool call, not most of them.

PreToolUse Hooks

PreToolUse hooks execute before a tool call runs. They receive the tool name and input parameters and can allow, block, or modify the call before it reaches the tool execution layer. A PreToolUse hook is the correct choice when the goal is to prevent a tool call from running unless certain conditions are met, for example, blocking a file deletion unless a backup confirmation exists or preventing a database write unless an authorization check has passed.

When a PreToolUse hook returns exit code 2, the tool is blocked even in `bypassPermissions` mode or with `--dangerously-skip-permissions`. This makes PreToolUse hooks the strongest enforcement mechanism available in Claude Code.

PostToolUse Hooks

PostToolUse hooks execute after a tool call completes. They receive the tool name, the input that was used, and the output the tool produced. They can inspect the output, transform it, log it, or trigger follow-up actions. A PostToolUse hook is the correct choice when the goal is to ensure something happens after every tool invocation, for example, running a linter after every file edit, validating a tool output schema before returning it to Claude, or appending an audit log entry after every write operation.

A common PostToolUse configuration runs ESLint after every file modification using a matcher that targets Edit and Write tool calls. The hook fires unconditionally every time either tool completes, regardless of what Claude decided to do with the file.

Hook Matchers

A hook matcher specifies which tool calls the hook applies to. Matchers allow developers to write focused hooks that target specific tools rather than intercepting every tool call in the session.

- A matcher targeting a single tool name, for example, `Edit` applies only to Edit tool calls.
- A pipe-separated list, for example, `Edit|Write` applies to both Edit and Write calls.
- An empty matcher applies to all tool calls in the session.

Correct matcher design is tested on the CCA-F exam. Overly broad matchers can introduce performance overhead and unintended side effects. Overly narrow matchers may miss the tool calls they were intended to intercept. The correct design matches the enforcement scope: if a linting rule must apply to every file modification operation, the matcher should cover all file-writing tools.

Using Hooks for Data Normalization

PostToolUse hooks are especially useful for data normalization, the process of transforming raw tool output into a consistent format before Claude processes it. This has two benefits: it reduces the cognitive

load on the model by removing irrelevant fields, and it ensures that downstream processing logic receives consistently formatted inputs regardless of which version of a tool or external service produced the output.

A normalization hook might strip unnecessary fields from a large API response, convert timestamps to a standard timezone, normalize status codes to a defined vocabulary, or extract the relevant portion of a large document chunk before returning it to Claude's context. The goal is to reduce noise, improve consistency, and prevent context bloat from verbose raw outputs.

Hooks vs. CLAUDE.md — The Enforcement Spectrum

The CCA-F exam frequently presents scenarios in which a team has written a CLAUDE.md rule requiring certain behavior and has found that the rule is followed most of the time but not always. The correct diagnostic is CLAUDE.md. Instructions are advisory. Claude processes them as context and generally follows them, but they are not binding at the system level. Strengthening the wording, adding "IMPORTANT" or "ALWAYS," may reduce violations but cannot eliminate them.

Hooks occupy the other end of the enforcement spectrum. They execute unconditionally at the application layer and cannot be bypassed by prompt context, model reasoning, or session-level instructions. When a rule must be applied on every tool call without exception, it belongs in a hook.

Path-scoped rules and skills also exist between these two extremes, but they are still advisory. They load context conditionally and improve relevance, but they share the same compliance properties as CLAUDE.md: they guide the model rather than enforce behavior programmatically.

EXAM TIP: The advisory-versus-deterministic distinction is one of the highest-frequency patterns in Domain 1 and Domain 3 questions. Any scenario that describes intermittent rule violations despite clear CLAUDE.md instructions is signaling that a hook is needed. Any scenario that asks which mechanism guarantees consistent enforcement is testing this distinction directly.

G. Task Decomposition Strategies

Task decomposition is the process of breaking a complex goal into smaller units of work that can be assigned to subagents or handled in separate passes. It is one of the most important design skills in multi-agent architecture because the quality of decomposition directly determines the quality of the final output. A coordinator that decomposes well produces complete, non-overlapping subtasks that collectively cover the goal. A coordinator that decomposes poorly produces gaps, overlaps, or subtasks that cannot be independently verified.

The CCA-F exam tests decomposition as a judgment skill, not a mechanical one. Candidates are expected to evaluate a given decomposition and identify whether it covers the problem adequately, whether its partitions are genuinely independent, and whether the coordinator has the information it needs to verify completion.

Decomposition Dimensions

Tasks can be decomposed along several dimensions. The correct dimension depends on the nature of the task, not on convenience.

Domain decomposition splits a broad topic into subject-matter slices. For a research task on the impact of AI on industries, domain decomposition produces partitions such as music, writing, film, visual arts, and commerce, not vague partitions such as "online" and "offline" that do not correspond to the structure of the subject matter. The exam guide uses the creative industries example to illustrate the failure of domain-shallow decomposition: if the coordinator decomposes into graphic design, digital art, and photography, the final synthesis omits music, writing, and film entirely. The decomposition appeared complete at design time but was not collectively sufficient.

Source-type decomposition splits a task by the type of evidence required. A research coordinator might send one subagent to gather current web sources, another to analyze internal documents, and another to query a structured database. Each subagent specializes in accessing and interpreting one type of source, and the coordinator synthesizes across all three. This avoids context window saturation by keeping raw gathering work isolated from synthesis reasoning.

Functional decomposition splits a task by stage of processing. A document extraction pipeline might decompose into a retrieval stage, a parsing stage, a validation stage, and a formatting stage. Each stage performs one function and passes a structured handoff to the next. Functional decomposition is most appropriate for sequential pipelines where each stage's output is a prerequisite for the next.

Parallelism vs. Sequentiality

Not all decompositions are parallel. Some subtasks must wait for the results of others before they can begin. The coordinator is responsible for understanding these dependencies and scheduling subtasks accordingly.

Parallel decomposition is appropriate when subtasks are independent: each can proceed with only the information in its own context handoff, and its output does not depend on any other subagent's result. Parallel execution reduces total wall-clock time and is the preferred pattern for investigative or research workflows where multiple evidence streams can be gathered simultaneously.

Sequential decomposition is appropriate when one stage's output is required as input for the next. A verification stage must complete before an access stage begins. A data gathering stage must be complete before a synthesis stage can run. Forcing sequential stages to run in parallel will cause the downstream stage to hallucinate inputs it has not yet received.

Mixed workflows combine both: independent subtasks run in parallel within each phase, and phases are sequenced by dependency. The coordinator manages the phase boundaries and ensures that handoffs between phases are complete before the next phase begins.

Verifying Decomposition Completeness

A correctly decomposed task has two properties: distinctness and collective sufficiency. Distinctness means each subtask covers a portion of the problem that no other subtask covers. Collective sufficiency means the subtasks together cover the entire problem. A gap in coverage means a dimension of the problem is never investigated, and the final synthesis will silently omit it.

The most practical way for a coordinator to verify collective sufficiency is to evaluate the synthesis output against the original task statement before returning it to the user. If a requested dimension is missing or thinly supported, the coordinator should identify which subagent was responsible for that area and either re-delegate a targeted follow-up or acknowledge the gap.

When Not to Decompose

Decomposition introduces coordination overhead. Each subagent requires a context handoff, produces a result that must be parsed and integrated, and adds latency to the overall workflow. For simple tasks where a single agent has sufficient context and tools, decomposition provides no benefit and increases complexity.

The correct threshold for decomposition is when the task requires either context isolation, where the inputs and reasoning for one subtask would pollute or exhaust the context window if handled by the main agent, or specialization, where different subtasks require different tools, models, or instructional contexts that cannot be combined in a single agent configuration.

EXAM TIP: If a scenario describes a coordinator that produced an incomplete or imbalanced final synthesis, the root cause is almost always decomposition design. The fix is to redesign the decomposition so that partitions are genuinely distinct and collectively sufficient, not to instruct the coordinator more firmly.

Resources

- https://docs.anthropic.com/en/docs/claude-code/sub-agents
- https://docs.anthropic.com/en/docs/agents-and-tools/agent-sdk
- https://docs.anthropic.com/en/docs/claude-code/overview

H. Session State, Resumption & Forking

In simple single-turn interactions, session state is not a concern. Claude receives a prompt, produces a response, and the interaction is complete. In agentic workflows, especially long-running ones involving codebase exploration, multi-phase research, or iterative development, session state is an architectural concern. The agent must maintain continuity across turns, recover from interruptions, and sometimes explore multiple approaches without losing the work invested in prior steps.

The CCA-F exam tests session state management as both a conceptual and a practical skill. Candidates are expected to understand the mechanisms available for preserving, resuming, and forking sessions, as well as when each mechanism is appropriate.

Session Resumption

A session can be resumed by name when a prior session was named explicitly using the `--resume` flag or a session naming convention. Resumption allows the agent to continue from the last saved state without reprocessing the conversation history from scratch. This is particularly useful for codebase exploration tasks that have already completed an initial survey phase, research workflows that have gathered evidence in a prior session, and iterative development workflows where prior context reduces redundant tool calls.

The important distinction is between resuming a session and starting a new session with context manually re-injected. Session resumption uses the stored conversation history and tool results directly. Context re-injection requires the developer to summarize and pass the relevant state in the new session's initial prompt, which is less efficient and more prone to omission. When a task is too large to complete in

a single session and continuity of prior tool results is important, session resumption is the correct pattern.

Session Forking

Session forking creates a copy of the current session at a specific point in time and continues two separate explorations from that point. Forking is appropriate when two or more implementation approaches need to be evaluated in parallel, when the next step in a workflow is uncertain and both branches need to be explored before a decision is made, or when a risky operation needs to be tested in an isolated copy before being applied to the main session.

Forking preserves the shared history and tool results up to the fork point, which eliminates the need to re-gather context that both branches require. After the fork, each branch accumulates its own additional tool calls and results independently. The coordinator or developer can then compare the outcomes of both branches and select the better result.

A common exam pattern is a scenario where an engineer needs to try two different refactoring approaches to a codebase without committing to either. The correct answer is "session forking," not creating two separate sessions from scratch. Creating separate sessions requires re-establishing shared context from the beginning, while forking preserves the already-gathered state and begins the divergence at the decision point.

Scratchpad Files for Long Sessions

The context window is finite. In a long-running agentic session, tool results, file contents, and conversation history accumulate and eventually approach or exceed the available context. When context fills, earlier information is displaced, and the agent may lose track of findings or decisions made earlier in the session.

Scratchpad files are a practical pattern for preserving session state across context limits. The agent writes its intermediate findings, decisions, and working notes to a file as it progresses. When context becomes constrained, the agent can use the `/clear` command to reset the conversation context and then re-read the scratchpad file to restore its working state. This allows indefinitely long workflows to proceed without losing the substance of prior work.

For exam purposes, the scratchpad pattern is the correct answer for scenarios involving long-running tasks that must survive context resets. Relying on session resumption alone does not address context window exhaustion within a session. Scratchpad files solve the within-session context saturation problem.

Sub-Agent Spawning for Context Management

An alternative to scratchpad files is spawning a subagent for context-intensive sub-tasks. Instead of accumulating all tool results in the main session, the coordinator spawns a subagent to perform a bounded investigation, receives only the subagent's final output, and keeps the main session's context focused on coordination-level reasoning rather than raw evidence.

This is preferable to scratchpad files when the subtask involves reading many large files, running verbose commands, or performing multi-step searches that would saturate the main context. The subagent's

internal processing, including all its intermediate tool calls and results, remains isolated in its own session, and only the structured summary returns to the coordinator.

The /clear Command

The `/clear` command resets the current session's conversation context without terminating the session. It is appropriate when the accumulated context from prior turns is no longer needed and would only consume context window space. After `/clear`, the agent starts fresh from the system prompt, with no memory of prior tool calls unless they were preserved in a scratchpad file or a structured external artifact.

The important point: `/clear` solves context saturation but destroys continuity unless the agent has proactively written its working state to a persistent artifact. `/clear` without a scratchpad results in the agent restarting from scratch. `/clear` with a scratchpad allows the agent to restore its working state and continue making meaningful progress.

EXAM TIP: When a scenario involves a long-running task that is degrading in quality or becoming incoherent, the root cause is usually context saturation. The correct response depends on the type of task: if the task can be broken into bounded sub-tasks, spawn subagents. If the task must continue in a single session, combine `/clear` with a scratchpad file pattern.

Resources

- https://docs.anthropic.com/en/docs/claude-code/overview
- https://docs.anthropic.com/en/docs/agents-and-tools/agent-sdk
- https://docs.anthropic.com/en/docs/claude-code/sub-agents

Domain 1: Agentic Architecture & Orchestration — Sample Questions

Question 1

Agents must interface with external data sources securely. The correct integration principle is:

1. Expose API keys in prompts.
2. Use secure connectors with scoped access.
3. Avoid external data entirely.
4. Only rely on user uploads.

Correct Answer: 2

Explanation:

Connectors allow Claude to access your apps and services, retrieve your data, and perform actions within those connected services. Claude inherits the permissions of each individual from the connected service. If someone is unable to access a specific file, channel, or record in the source system, then the connector will also be unable to access it through Claude.

Security is maintained by using scoped, authenticated connectors rather than including sensitive credentials inside the prompt text.

Using secure connectors with scoped access is the ideal integration principle for securely interfacing with external data sources. Secure connectors ensure that only authorized agents can access the data, and scoped access limits the data that can be retrieved, reducing the risk of data breaches.

Hence, the correct answer is: **Use secure connectors with scoped access.**

The option that says: *Expose API keys in prompts* is incorrect because exposing API keys in prompts is a security risk, as it can typically lead to unauthorized access to external data sources. It is not a recommended integration principle for securely interfacing with external data sources.

The option that says: *Avoid external data entirely* is incorrect because avoiding external data entirely may not always be feasible or practical, especially in scenarios where external data sources are essential for the functionality of the system. It is not a recommended integration principle as it may limit the capabilities of the system.

The option that says: *Only rely on user uploads* is incorrect because relying only on user uploads for data integration is not a secure or reliable method. User uploads can introduce security vulnerabilities and may not provide real-time access to external data sources. It is not a recommended integration principle for securely interfacing with external data sources.

References:

- https://support.claude.com/en/articles/12684923-microsoft-365-connector-security-guide
- https://support.claude.com/en/articles/11176164-use-connectors-to-extend-claude-s-capabilities

Question 2

A team wants Claude to remember prior outputs within a session. The correct architectural component is:

1. Stateless prompts
2. Persistent context storage and retrieval
3. One-shot summarization prompt
4. Manual transcript copy

Correct Answer: 2

Explanation:

Remembering prior outputs requires state management through persistent storage (like a scratchpad or database) to retrieve context in subsequent turns.

Persistent context storage and retrieval allows Claude to store and recall information from previous interactions within a session. This architectural component enables Claude to remember prior outputs, making it the correct choice for the team's requirement.

Hence, the correct answer is: **Persistent context storage and retrieval.**

Stateless prompts is incorrect because stateless prompts do not retain any information or context from previous interactions within a session. They are primarily designed to generate responses based solely on the current input without any memory of past outputs. This choice does not align with the requirement of remembering prior outputs within a session.

One-shot summarization prompt is incorrect because one-shot summarization prompts are designed to provide a concise summary or response based on a single input without the need to remember or retain context from previous interactions. This choice does not support the team's goal of having Claude remember prior outputs within a session.

Manual transcript copy is incorrect because manual transcript copy typically involves manually copying and storing transcripts or outputs from each session, which is not an efficient or scalable solution for remembering prior outputs within a session. This choice does not provide an automated mechanism for Claude to retain and recall information from previous interactions.

References:

- https://platform.claude.com/docs/en/managed-agents/memory
- https://platform.claude.com/docs/en/agents-and-tools/tool-use/memory-tool

---

References for Domain 1: Agentic Architecture & Orchestration

Foundations of Agentic Architecture

- https://docs.anthropic.com/en/docs/agents-and-tools/agent-sdk
- https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/overview
- https://docs.anthropic.com/en/docs/claude-code/sub-agents

Agentic Loop Lifecycle

- https://docs.anthropic.com/en/docs/agents-and-tools/agent-sdk
- https://docs.anthropic.com/en/api/messages
- https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/overview

Coordinator-Subagent Orchestration

- https://docs.anthropic.com/en/docs/claude-code/sub-agents
- https://docs.anthropic.com/en/docs/agents-and-tools/agent-sdk
- https://docs.anthropic.com/en/docs/claude-code/overview

Subagent Invocation & Context Passing

- https://docs.anthropic.com/en/docs/claude-code/sub-agents
- https://docs.anthropic.com/en/docs/agents-and-tools/tool-use
- https://docs.anthropic.com/en/docs/agents-and-tools/agent-sdk

Multi-Step Workflow Enforcement & Handoff

- https://docs.anthropic.com/en/docs/claude-code/hooks-guide
- https://docs.anthropic.com/en/docs/agents-and-tools/agent-sdk
- https://docs.anthropic.com/en/docs/claude-code/memory

Agent SDK Hooks for Tool Interception & Data Normalization

- https://docs.anthropic.com/en/docs/claude-code/hooks-guide
- https://docs.anthropic.com/en/docs/claude-code/memory

- https://docs.anthropic.com/en/docs/agents-and-tools/agent-sdk

Task Decomposition Strategies

- https://docs.anthropic.com/en/docs/claude-code/sub-agents
- https://docs.anthropic.com/en/docs/agents-and-tools/agent-sdk
- https://docs.anthropic.com/en/docs/claude-code/overview

Session State, Resumption & Forking

- https://docs.anthropic.com/en/docs/claude-code/overview
- https://docs.anthropic.com/en/docs/agents-and-tools/agent-sdk
- https://docs.anthropic.com/en/docs/claude-code/sub-agents